

hwk2/tb6_gbr_branch_system.vhd

```

1  -----
2  --  TB6_GBR_Branch_System  --  SH-2 CPU Integration Testbench 6
3  --
4  --  Tests GBR byte/word/longword ops, byte read-modify-write @(R0,GBR),
5  --  TAS.B, delay-slot branches (BT/S, BF/S), JMP, JSR, BSRF, VBR ops. PC
6  --  relative offsets will need to be adjusted when pipelining.
7  --
8  --  Instructions tested (21 checks):
9  --      MOV.B R0,@(disp,GBR)      MOV.B @(disp,GBR),R0
10 --      MOV.W R0,@(disp,GBR)      MOV.W @(disp,GBR),R0
11 --      MOV.L R0,@(disp,GBR)      MOV.L @(disp,GBR),R0
12 --      AND.B #imm,@(R0,GBR)      OR.B #imm,@(R0,GBR)
13 --      XOR.B #imm,@(R0,GBR)      TST.B #imm,@(R0,GBR)
14 --      TAS.B @Rn
15 --      BT/S label (taken + not taken)
16 --      BF/S label (taken + not taken)
17 --      JMP @Rm                    JSR @Rm
18 --      BSRF Rm
19 --      LDC Rm,VBR                  STC VBR,Rn
20 --      STC.L VBR,@-Rn              LDC.L @Rm+,VBR
21 --
22 --  Disassembly (address : encoding : instruction):
23 --      0x0000: DE4A  MOV.L @(0x4A,PC),R14
24 --      0x0002: DF4B  MOV.L @(0x4B,PC),R15
25 --      0x0004: 0009  NOP
26 --      0x0006: D14B  MOV.L @(0x4B,PC),R1
27 --      0x0008: 411E  LDC R1,GBR                GBR = 0x1200
28 --      0x000A: E05A  MOV #0x5A,R0
29 --      0x000C: C000  MOV.B R0,@(0,GBR)          [0x1200] byte = 0x5A
30 --      0x000E: C400  MOV.B @(0,GBR),R0          R0 = sign-ext 0x5A
31 --      0x0010: 2E02  MOV.L R0,@R14
32 --      0x0012: 7E04  ADD #4,R14
33 --      0x0014: D048  MOV.L @(0x48,PC),R0
34 --      0x0016: C102  MOV.W R0,@(2*2,GBR)        [0x1204] word = 0x1234
35 --      0x0018: C502  MOV.W @(2*2,GBR),R0        R0 = sign-ext 0x1234
36 --      0x001A: 2E02  MOV.L R0,@R14
37 --      0x001C: 7E04  ADD #4,R14
38 --      0x001E: D047  MOV.L @(0x47,PC),R0
39 --      0x0020: C202  MOV.L R0,@(2*4,GBR)        [0x1208] = 0xCAFEBAFE
40 --      0x0022: C602  MOV.L @(2*4,GBR),R0        R0 = 0xCAFEBAFE
41 --      0x0024: 2E02  MOV.L R0,@R14
42 --      0x0026: 7E04  ADD #4,R14
43 --      0x0028: E010  MOV #0x10,R0                R0 = 0x10 offset
44 --      0x002A: E1FF  MOV #0xFF,R1
45 --      0x002C: D344  MOV.L @(0x44,PC),R3
46 --      0x002E: 2310  MOV.B R1,@R3                [0x1210] = 0xFF
47 --      0x0030: CD0F  AND.B #0x0F,@(R0,GBR) --> 0xFF & 0x0F = 0x0F
48 --      0x0032: 6230  MOV.B @R3,R2                R2 = sign-ext 0x0F

```

```

49  -- 0x0034: 2E22 MOV.L R2,@R14
50  -- 0x0036: 7E04 ADD #4,R14
51  -- 0x0038: E010 MOV #0x10,R0
52  -- 0x003A: CFA0 OR.B #0xA0,@(R0,GBR) -> 0x0F | 0xA0 = 0xAF
53  -- 0x003C: 6230 MOV.B @R3,R2          R2 = sign-ext 0xAF
54  -- 0x003E: 2E22 MOV.L R2,@R14
55  -- 0x0040: 7E04 ADD #4,R14
56  -- 0x0042: E010 MOV #0x10,R0
57  -- 0x0044: CEFF XOR.B #0xFF,@(R0,GBR) -> 0xAF ^ 0xFF = 0x50
58  -- 0x0046: 6230 MOV.B @R3,R2          R2 = sign-ext 0x50
59  -- 0x0048: 2E22 MOV.L R2,@R14
60  -- 0x004A: 7E04 ADD #4,R14
61  -- 0x004C: E010 MOV #0x10,R0
62  -- 0x004E: CC0F TST.B #0x0F,@(R0,GBR) T = (0x50 & 0x0F == 0) = 1
63  -- 0x0050: 0129 MOVT R1
64  -- 0x0052: 2E12 MOV.L R1,@R14
65  -- 0x0054: 7E04 ADD #4,R14
66  -- 0x0056: E100 MOV #0,R1
67  -- 0x0058: D33A MOV.L @(0x3A,PC),R3
68  -- 0x005A: 2310 MOV.B R1,@R3          [0x1214] = 0x00
69  -- 0x005C: 431B TAS.B @R3             T=(old==0)=1; set bit7
70  -- 0x005E: 0129 MOVT R1               R1 = 1
71  -- 0x0060: 2E12 MOV.L R1,@R14
72  -- 0x0062: 7E04 ADD #4,R14
73  -- 0x0064: 6230 MOV.B @R3,R2          R2 = sign-ext 0x80
74  -- 0x0066: 2E22 MOV.L R2,@R14
75  -- 0x0068: 7E04 ADD #4,R14
76  -- 0x006A: 0018 SETT                  T = 1
77  -- 0x006C: E109 MOV #9,R1
78  -- 0x006E: 8D01 BT/S +1              taken
79  -- 0x0070: 7101 ADD #1,R1            delay slot: R1 = 10
80  -- 0x0072: E1EE MOV #0xEE,R1         (poison - skipped)
81  -- 0x0074: 2E12 MOV.L R1,@R14        [R14] = 10
82  -- 0x0076: 7E04 ADD #4,R14
83  -- 0x0078: 0008 CLRT                  T = 0
84  -- 0x007A: E10B MOV #11,R1
85  -- 0x007C: 8D01 BT/S +1              not taken
86  -- 0x007E: 0009 NOP                  delay slot
87  -- 0x0080: 7101 ADD #1,R1            R1 = 12 (fall through)
88  -- 0x0082: 2E12 MOV.L R1,@R14        [R14] = 12
89  -- 0x0084: 7E04 ADD #4,R14
90  -- 0x0086: 0008 CLRT                  T = 0
91  -- 0x0088: E10D MOV #13,R1
92  -- 0x008A: 8F01 BF/S +1              taken
93  -- 0x008C: 7101 ADD #1,R1            delay slot: R1 = 14
94  -- 0x008E: E1EE MOV #0xEE,R1         (poison - skipped)
95  -- 0x0090: 2E12 MOV.L R1,@R14        [R14] = 14
96  -- 0x0092: 7E04 ADD #4,R14
97  -- 0x0094: 0018 SETT                  T = 1
98  -- 0x0096: E10F MOV #15,R1

```

```

99  --- 0x0098: 8F01 BF/S +1          not taken
100 --- 0x009A: 0009 NOP              delay slot
101 --- 0x009C: 7101 ADD #1,R1        R1 = 16 (fall through)
102 --- 0x009E: 2E12 MOV.L R1,@R14    [R14] = 16
103 --- 0x00A0: 7E04 ADD #4,R14
104 --- 0x00A2: D128 MOV.L @(0x29,PC),R1
105 --- 0x00A4: E211 MOV #17,R2
106 --- 0x00A6: 412B JMP @R1
107 --- 0x00A8: 7201 ADD #1,R2        delay slot: R2 = 18
108 --- 0x00AA: E2EE MOV #0xEE,R2     (poison)
109 --- 0x00AC: 2E22 MOV.L R2,@R14    [R14] = 18
110 --- 0x00AE: 7E04 ADD #4,R14
111 --- 0x00B0: D126 MOV.L @(0x26,PC),R1
112 --- 0x00B2: E213 MOV #19,R2
113 --- 0x00B4: 410B JSR @R1          PR = PC+4
114 --- 0x00B6: 0009 NOP              delay slot
115 --- 0x00B8: E214 MOV #20,R2      after RTS return
116 --- 0x00BA: A004 BRA +4
117 --- 0x00BC: 0009 NOP              delay slot
118 --- 0x00BE: 2E22 MOV.L R2,@R14    sub: store R2=19
119 --- 0x00C0: 7E04 ADD #4,R14
120 --- 0x00C2: 000B RTS
121 --- 0x00C4: 0009 NOP              delay slot
122 --- 0x00C6: 2E22 MOV.L R2,@R14    [R14] = 20 (return val)
123 --- 0x00C8: 7E04 ADD #4,R14
124 --- 0x00CA: E215 MOV #21,R2
125 --- 0x00CC: E106 MOV #6,R1
126 --- 0x00CE: 0103 BSRF R1          -> sub; PR = PC+4
127 --- 0x00D0: 0009 NOP              delay slot
128 --- 0x00D2: E216 MOV #22,R2      after BSRF return
129 --- 0x00D4: A004 BRA +4
130 --- 0x00D6: 0009 NOP              delay slot
131 --- 0x00D8: 2E22 MOV.L R2,@R14    BSRF sub: store R2=21
132 --- 0x00DA: 7E04 ADD #4,R14
133 --- 0x00DC: 000B RTS
134 --- 0x00DE: 0009 NOP              delay slot
135 --- 0x00E0: 2E22 MOV.L R2,@R14    [R14] = 22 (return)
136 --- 0x00E2: 7E04 ADD #4,R14
137 --- 0x00E4: D11A MOV.L @(0x1A,PC),R1
138 --- 0x00E6: 412E LDC R1,VBR        VBR = 0x00002000
139 --- 0x00E8: 0222 STC VBR,R2       R2 = 0x00002000
140 --- 0x00EA: 2E22 MOV.L R2,@R14
141 --- 0x00EC: 7E04 ADD #4,R14
142 --- 0x00EE: 4F23 STC.L VBR,@-R15  push VBR
143 --- 0x00F0: 62F2 MOV.L @R15,R2    peek stack
144 --- 0x00F2: 7F04 ADD #4,R15       restore SP
145 --- 0x00F4: 2E22 MOV.L R2,@R14
146 --- 0x00F6: 7E04 ADD #4,R14
147 --- 0x00F8: D116 MOV.L @(0x16,PC),R1
148 --- 0x00FA: 2F16 MOV.L R1,@-R15  push 0x3000

```

```

149  --      0x00FC: 63F3  MOV R15,R3
150  --      0x00FE: 4327  LDC.L @R3+,VBR      VBR = 0x00003000
151  --      0x0100: 7F04  ADD #4,R15          restore SP
152  --      0x0102: 0222  STC VBR,R2
153  --      0x0104: 2E22  MOV.L R2,@R14
154  --      0x0106: 7E04  ADD #4,R14
155  --      0x0108: A000  BRA .                self-loop
156  --      0x010A: 0009  NOP                delay slot
157  --      -- Data constants (at 0x0128) --
158  --      0x0128: 00001000
159  --      0x012C: 00001100
160  --      0x0130: 00001200
161  --      0x0134: 00001234
162  --      0x0138: CAFEBABE
163  --      0x013C: 00001210
164  --      0x0140: 00001214
165  --      0x0144: 000000AC  (JMP target)
166  --      0x0148: 000000BE  (JSR target)
167  --      0x014C: 00002000
168  --      0x0150: 00003000

```

169

170

171 **library** ieee;172 **use** ieee.std_logic_1164.all;173 **use** ieee.numeric_std.all;174 **use** std.textio.all;

175

176 **entity** TB6_GBRBranchSystem **is**177 **end entity**;

178

179 **architecture** sim **of** TB6_GBRBranchSystem **is**

180

181 **constant** CLK_PERIOD : **time** **:=** 20 ns;182 **constant** PROG_HEX : **string** **:=** "tb6_program.hex";183 **constant** MEM_WORDS : **integer** **:=** 256;

184

185 **signal** clk : **std_logic** **:=** '0';186 **signal** reset : **std_logic** **:=** '0';187 **signal** loading : **std_logic** **:=** '1';

188

189 -- Shared bus

190 **signal** AB : **std_logic_vector**(31 **downto** 0);191 **signal** DB : **std_logic_vector**(31 **downto** 0);192 **signal** RE0, RE1, RE2, RE3 : **std_logic**;193 **signal** WE0, WE1, WE2, WE3 : **std_logic**;

194

195 -- TB-side bus signals

196 **signal** tb_AB : **std_logic_vector**(31 **downto** 0) **:=** (others => '0');197 **signal** tb_DB : **std_logic_vector**(31 **downto** 0) **:=** (others => '0');198 **signal** tb_RE : **std_logic** **:=** '1';

```

199     signal tb_WE : std_logic := '1';
200
201 begin
202
203     clk <= not clk after CLK_PERIOD / 2;
204
205     AB <= tb_AB when loading = '1' else (others => 'Z');
206     DB <= tb_DB when (loading = '1' and tb_WE = '0') else (others => 'Z');
207     RE0 <= tb_RE when loading = '1' else 'Z';
208     RE1 <= tb_RE when loading = '1' else 'Z';
209     RE2 <= tb_RE when loading = '1' else 'Z';
210     RE3 <= tb_RE when loading = '1' else 'Z';
211     WE0 <= tb_WE when loading = '1' else 'Z';
212     WE1 <= tb_WE when loading = '1' else 'Z';
213     WE2 <= tb_WE when loading = '1' else 'Z';
214     WE3 <= tb_WE when loading = '1' else 'Z';
215
216     dut : entity work.SH2_CPU
217         port map (
218             Reset => reset, NMI => '1', INT => '1', clock => clk,
219             AB => AB, DB => DB,
220             RE0 => RE0, RE1 => RE1, RE2 => RE2, RE3 => RE3,
221             WE0 => WE0, WE1 => WE1, WE2 => WE2, WE3 => WE3
222         );
223
224     mem : entity work.MEMORY32x32
225         generic map (
226             MEMSIZE => MEM_WORDS,
227             START_ADDR0 => 16#00000000#,
228             START_ADDR1 => 16#00001000#,
229             START_ADDR2 => 16#10000000#,
230             START_ADDR3 => 16#20000000#
231         )
232         port map (
233             RE0 => RE0, RE1 => RE1, RE2 => RE2, RE3 => RE3,
234             WE0 => WE0, WE1 => WE1, WE2 => WE2, WE3 => WE3,
235             MemAB => AB, MemDB => DB
236         );
237
238     stim : process
239         file hex_file : text;
240         variable fstatus : file_open_status;
241         variable l : line;
242         variable word : std_logic_vector(31 downto 0);
243         variable addr : unsigned(31 downto 0);
244         variable pass_cnt : integer := 0;
245         variable fail_cnt : integer := 0;
246
247         procedure check(a : std_logic_vector(31 downto 0);
248             exp : std_logic_vector(31 downto 0);

```

```

249         tag : string) is
250     begin
251         tb_AB <= a;
252         tb_RE <= '0';
253         tb_WE <= '1';
254         wait for CLK_PERIOD;
255         if DB = exp then
256             pass_cnt := pass_cnt + 1;
257             report "PASS: " & tag severity note;
258         else
259             fail_cnt := fail_cnt + 1;
260             report "FAIL: " & tag
261                 & " got=0x" & to_hstring(DB)
262                 & " exp=0x" & to_hstring(exp) severity error;
263         end if;
264         tb_RE <= '1';
265         wait for CLK_PERIOD;
266     end procedure;
267
268 begin
269     -----
270     -- LOAD PHASE
271     -----
272     loading <= '1';
273     reset   <= '0';
274     tb_RE   <= '1';
275     tb_WE   <= '1';
276     wait for CLK_PERIOD * 2;
277
278     file_open(fstatus, hex_file, PROG_HEX, read_mode);
279     assert fstatus = open_ok
280         report "Cannot open " & PROG_HEX severity failure;
281
282     addr := (others => '0');
283     while not endfile(hex_file) loop
284         readline(hex_file, l);
285         if l'length > 0 then
286             hread(l, word);
287             tb_AB <= std_logic_vector(addr);
288             tb_DB <= word;
289             tb_WE <= '0';
290             wait for CLK_PERIOD;
291             tb_WE <= '1';
292             wait for CLK_PERIOD;
293             addr := addr + 4;
294         end if;
295     end loop;
296     file_close(hex_file);
297
298     -----

```

```

299  -- CPU PHASE
300  -----
301  wait until rising_edge(clk);
302  loading <= '0';
303  reset   <= '1';
304
305  wait for CLK_PERIOD * 2;
306
307  for i in 1 to 500 loop
308      wait until rising_edge(clk);
309  end loop;
310
311  -----
312  -- CHECK PHASE
313  -----
314  wait until rising_edge(clk);
315  reset   <= '0';
316  loading <= '1';
317  tb_RE   <= '1';
318  tb_WE   <= '1';
319  wait for CLK_PERIOD * 2;
320
321  check(x"00001000", x"0000005A", "MOV.B @(disp,GBR) roundtrip");
322  check(x"00001004", x"00001234", "MOV.W @(disp,GBR) roundtrip");
323  check(x"00001008", x"CAFEBAFE", "MOV.L @(disp,GBR) roundtrip");
324  check(x"0000100C", x"0000000F", "AND.B #imm,@(R0,GBR)");
325  check(x"00001010", x"FFFFFFFA", "OR.B #imm,@(R0,GBR)");
326  check(x"00001014", x"00000050", "XOR.B #imm,@(R0,GBR)");
327  check(x"00001018", x"00000001", "TST.B #imm,@(R0,GBR)");
328  check(x"0000101C", x"00000001", "TAS.B @Rn (T result)");
329  check(x"00001020", x"FFFFFFF8", "TAS.B @Rn (byte got bit7)");
330  check(x"00001024", x"0000000A", "BT/S taken + delay slot");
331  check(x"00001028", x"0000000C", "BT/S not taken");
332  check(x"0000102C", x"0000000E", "BF/S taken + delay slot");
333  check(x"00001030", x"00000010", "BF/S not taken");
334  check(x"00001034", x"00000012", "JMP @Rm + delay slot");
335  check(x"00001038", x"00000013", "JSR @Rm (sub body ran)");
336  check(x"0000103C", x"00000014", "JSR+RTS (return reached)");
337  check(x"00001040", x"00000015", "BSRF Rm (sub body)");
338  check(x"00001044", x"00000016", "BSRF+RTS (return reached)");
339  check(x"00001048", x"00002000", "LDC Rm,VBR + STC VBR,Rn");
340  check(x"0000104C", x"00002000", "STC.L VBR,@-Rn");
341  check(x"00001050", x"00003000", "LDC.L @Rm+,VBR");
342
343  -----
344  -- SUMMARY
345  -----
346  report "TB6: PASSED=" & integer'image(pass_cnt)
347        & "   FAILED=" & integer'image(fail_cnt) severity note;
348  if fail_cnt = 0 then

```

```
349         report "*** ALL TESTS PASSED ***" severity note;
350     else
351         assert false
352             report integer'image(fail_cnt) & " TEST(S) FAILED"
353             severity failure;
354     end if;
355     wait;
356 end process;
357
358 end architecture;
359
```